

Manual técnico — RedEconómica v1.0.0

Documento para quien vaya a compilar, mantener o ampliar la aplicación.

1. Stack y requisitos

Elemento	Versión
Kotlin	2.0.21 (compilador de Compose vía <code>org.jetbrains.kotlin.plugin.compose</code>)
Android Gradle Plugin	8.5.2
Gradle Wrapper	8.7
JDK	17
compileSdk / targetSdk	34
minSdk	24
Compose BOM	2024.09.00
Material 3	vía BOM
Navigation Compose	2.8.0
Room	2.6.1 (procesador KSP 2.0.21-1.0.25)
Coroutines	1.8.1
Robolectric (pruebas)	4.13

Todas las versiones están fijadas en `gradle/libs.versions.toml`. **No se usa ninguna versión dinámica.**

Compilar

```
./gradlew clean
./gradlew testDebugUnitTest
./gradlew lintDebug
./gradlew assembleDebug
```

El APK sale en `app/build/outputs/apk/debug/app-debug.apk`.

Sin conexión no se puede compilar: Gradle necesita descargar AGP, Kotlin, Compose y Room de `dl.google.com` y Maven Central la primera vez.

2. Estructura del proyecto

```

RedEconomica/
├─ app/
│  └─ src/
│     └─ main/java/com/educalab/redeconomica/
│        └─ RedEconomicaApp.kt      Application + AppContainer (DI manual)
│        └─ MainActivity.kt         única Activity, todo Compose
│        └─ domain/
│           └─ model/               11 archivos: modelos puros
│              └─ engine/           11 archivos: los motores económicos
│                 └─ data/
│                    └─ local/       Room: entidades, DAOs, base, mappers, sembrador
│                       └─ repository/ 4 repositorios
│                          └─ seed/    contenido del Valle (datos)
│                             └─ ui/
│                                └─ theme/    colores, tipografía, formas
│                                   └─ art/    ilustraciones con Canvas
│                                      └─ components/ piezas reutilizables
│                                         └─ navigation/ rutas y NavHost
│                                            └─ screens/   pantallas + ViewModels
│   └─ test/                          136 pruebas JVM
│      └─ androidTest/                1 prueba instrumentada de humo
├─ database/    schema.sql · sample_data.sql
├─ docs/        documentación (+ docs/pdf)
├─ tools/       scripts Python de apoyo
└─ .github/workflows/build.yml

```

3. Arquitectura

MVVM + Repository, con una regla estricta: **la interfaz no calcula nada económico**.

```

Composable → ViewModel → EconomyEngine (domain) ← decide
                |
                └─ Repository → DAO → Room

```

- `domain/` no importa nada de `android.*` ni de Compose. Se puede probar en la JVM sin emulador, y así están hechas 119 de las 136 pruebas.
- `data/` traduce entre Room y el dominio (`Mappers`) y expone `Flow`.
- `ui/` observa `StateFlow` y dibuja.

Inyección de dependencias

`AppContainer` (en `RedEconomicaApp.kt`) construye la base de datos, los repositorios y los motores una sola vez. Los ViewModels se crean con `viewModelFactory { initializer { ... } }`. No se usa Hilt: el grafo es plano y un contenedor explícito es más legible y compila más rápido.

4. Los motores del dominio

`EconomyEngine` (fachada)

Reúne los motores y evalúa cualquier escenario:

```
fun evaluar(escenario: Scenario, respuesta: ScenarioAnswer, intentos: Int): AttemptResult
```

`ScenarioAnswer` es una interfaz sellada con una variante por mecánica (`Intercambio`, `Evaluacion`, `Especializar`, `Renuncia`, `Cooperar`, `Repartir`, `Decidir`, `Ordenar`). Además mantiene la **mesa de trueques** (`TradeSession` → `proponer()` → `TradeStep`), que es estado inmutable: cada propuesta devuelve una sesión nueva.

`TradeEngine`

Siete reglas, en este orden exacto:

1. La propuesta está completa (algo por algo) — `OFERTA_VACIA`
2. Quien propone tiene lo que entrega — `SIN_RECURSOS_PROPONENTE`
3. Quien recibe tiene lo que se le pide — `SIN_RECURSOS_RECEPTOR`
4. Lo ofrecido cubre alguna necesidad del receptor — `NO NECESITA LO OFRECIDO`
5. El receptor no pierde algo que necesita con urgencia ALTA y ya tenía cubierto — `PERDERIA LO QUE NECESITA`
6. $\text{valor}(\text{entrega}) * 100 \geq \text{valor}(\text{pide}) * 60$ — `DESEQUILIBRIO`
7. (opcional) Quien propone también cubre una necesidad — `SIN_BENEFICIO_MUTUO`

`valorBase` de cada recurso (1..4) es un **peso pedagógico**, nunca un precio, y no se muestra al niño. `buscarIntercambios()` enumera propuestas viables; se usa para pistas, para el Mercado y para validar escenarios.

SpecializationEngine

```
especializado en R → productividad[R]
"hace de todo"     → productividad[i] / k   (k = tareas que sabe; división entera)
sin tarea          → nada
```

`mejorPlan()` explora todas las combinaciones cuando caben (≤ 4096) y cae en una heurística voraz por ventaja relativa si no. `planesQueCumplen()` devuelve **todas** las soluciones válidas.

CooperationEngine

```
resultado          = min(capacidad de cada etapa)
capacidad(etapa)   = Σ rendimiento de quienes trabajan en ella
sin cooperar       = Σ min(rendimiento propio en todas las etapas)
```

Devuelve también los cuellos de botella y la mejora por cooperar.

ScarcityEngine

Valida repartos (nunca por encima de lo disponible; obliga a cubrir las urgencias ALTA) y decisiones con presupuesto. `repartosValidos()` y `decisionesPosibles()` enumeran todas las soluciones, con topes de búsqueda para no bloquear la interfaz.

OpportunityCostEngine

```
renuncia = cantidadElegida × productividad(renunciado) / productividad(elegido)
costo de R1 para X = productividad(X, R2) / productividad(X, R1)
```

Quien tiene el costo más bajo debería producir R1: esa es la ventaja comparativa, y el motor la explica con las dos cifras.

Otros

`ProductionChainEngine` (orden y punto de rotura, barajado reproducible por semilla), `LabEngine` (experimentos con producción, trueques automáticos y puesta en común), `ProgressEngine` (sellos, niveles, estados de misión, insignias), `DailyChallengeGenerator` (reto diario determinista por índice de día, sin relojes de servidor) y `ScenarioValidator`.

5. `Inventory`: la pieza que impide estados imposibles

```
class Inventory private constructor(val contenido: Map<String, Int>)
```

- No admite cantidades negativas (lanza `IllegalArgumentException`).
- Descarta las entradas en cero.
- `menos()` devuelve `null` cuando no hay suficiente, en vez de generar negativos.
- Es inmutable: toda operación devuelve un inventario nuevo.
- Se persiste como texto compacto: `"manzana:4|pan:2"`.

Ningún Composable modifica inventarios: todos pasan por los motores.

6. Persistencia

22 tablas Room, descritas en `docs/BASE_DE_DATOS.md` y reproducidas en `database/schema.sql`. Dos bloques: **catálogo** (sembrado al primer arranque por `DatabaseSeeder`, idempotente) y **estado** (lo que el niño hace).

Los contadores del perfil **no se guardan**: se calculan con `COUNT(*)` sobre las tablas de acciones y se combinan con `kotlinx.coroutines.flow.combine`. Es imposible que el progreso se desvincule de lo ocurrido.

`AppDatabase` usa `fallbackToDestructiveMigration()`: al no haber datos personales y ser el catálogo reproducible, un cambio de esquema vuelve a sembrar en lugar de arrastrar migraciones. Si en el futuro se quisiera conservar el progreso entre versiones, habría que añadir migraciones reales.

7. Interfaz

- **Tema** (`ui/theme/Theme.kt`): paleta del Valle, esquemas claro y oscuro, tipografía escalable (`textoGrande` multiplica por 1,18) y formas redondeadas.
- **Ilustraciones** (`ui/art/`): todo con `Canvas`. `ArtKit` define un sistema de coordenadas 0..100 (`lienzo { l -> ... }`) y primitivas (`circulo`, `poligono`, `caja`, `trazo`). `ResourceArt` dibuja los 20 recursos, `CharacterArt` los 9 personajes y 8 avatares, `ValleyArt` los 13 edificios y el paisaje, `BadgeArt` las 11 insignias y los 13 iconos de concepto. **Ninguna imagen se descarga ni se incrusta como binario.**
- **Componentes** (`ui/components/`): `TarjetaValle`, `ChipRecurso`, `BarraValle`, `FichaHabitante`, `EtiquetaUrgencia`, `EtiquetaEstado`, `TiloDice`, `PanelResultado`, `CabeceraActividad`, `SelectorCantidad`.
- **Navegación** (`ui/navigation/`): once rutas, `NavHost` único, destino inicial según `onboardingHecho`.
- **Pantallas**: el mapa (`PantallaValle`), la misión, la actividad (una sola pantalla que se transforma según `ScenarioPayload`), Mercado, Laboratorio, Diccionario, Almacén, Insignias, Perfil, Repaso y Onboarding.

Añadir una mecánica nueva

1. Añadir la variante a `ActivityKind` y a `ScenarioPayload`.
2. Escribir su motor en `domain/engine` (o ampliar uno existente).
3. Añadir la rama en `EconomyEngine.evaluar` y en `ScenarioValidator.validar`.
4. Añadir la variante de `ScenarioAnswer` y su método en `ActivityViewModel`.
5. Escribir el Composable y añadir la rama en `PantallaActividad`.
6. Añadir escenarios a `data/seed` y pruebas.

El compilador ayuda: los `when` sobre las clases selladas dejan de ser exhaustivos y fallan hasta que se cubren todas las ramas.

8. Contenido

`data/seed` contiene 20 recursos, 9 personajes (8 vecinos + Tilo), 42 escenarios repartidos en 14 misiones, 11 insignias, 24 objetos y 13 entradas de diccionario. `SeedContent.problemasDeIntegridad()` comprueba las referencias cruzadas y `ScenarioValidator` comprueba que cada escenario tiene solución; ambas cosas se ejecutan en las pruebas.

Los escenarios son **datos**, no código de interfaz: viven fuera de `ui/` y se validan antes de mostrarse. La ficha de cada escenario (id, misión, textos, dificultad) también está en Room, y de ahí salen los listados y el progreso.

9. Pruebas

```
./gradlew testDebugUnitTest           # 136 pruebas JVM
./gradlew connectedDebugAndroidTest    # 1 prueba instrumentada (requiere dispositivo)
```

`PersistenciaTest` usa **Robolectric 4.13** con Room en memoria: prueba sembrado, progreso, contadores, insignias, almacén, reto diario y reinicio con la base de datos real, sin emulador.

Herramientas de apoyo independientes del compilador:

```
python3 tools/verificar_escenarios.py   # reimplementa las reglas y valida los 42 desafíos
python3 tools/generar_sample_data.py    # regenera database/sample_data.sql desde el código
python3 tools/generate_launcher_icons.py # regenera los iconos PNG del lanzador
```

10. Integración continua

`.github/workflows/build.yml` ejecuta, en `ubuntu-latest` con JDK 17: `clean` → `testDebugUnitTest` → `lintDebug` → `assembleDebug`, renombra el APK a `RedEconomica-v1.0.0.apk`, calcula su SHA-256 y lo publica como artefacto junto con los informes de pruebas y de lint.

11. Decisiones y limitaciones conocidas

- **Sin Hilt** por tamaño del grafo (ver §3).
- **Sin migraciones Room** en la 1.0.0 (ver §6).
- **Arrastre implementado como tocar-para-colocar**, con alternativa de flechas y botones de cantidad: es la interacción real y además es accesible.
- **Sin archivos de sonido** en la 1.0.0: los interruptores de sonido y vibración existen y se guardan, pero la app es silenciosa.
- **Las reglas de los escenarios viven en `data/seed`**, no en columnas (justificado en la memoria, §13.2).
- **Compilación no verificada** en el entorno de generación: ver `docs/BUILD_REPORT.md`.